

Кто такой Bootstrap?

Bootstrap — это мощный, многофункциональный интерфейсный инструментарий.

Он содержит готовые шаблоны для кнопок, навигационных панелей, карточек, форм и множества других элементов, что позволяет быстро создавать современные и адаптивные сайты.

Где его найти и как скачать?

Есть несколько способов начать работу с Bootstrap:

- **Официальный сайт:** Самый простой способ — зайти на официальную страницу getbootstrap.com и нажать кнопку "Download".
- **CDN-ссылки (самый популярный способ):** Тебе даже не нужно скачивать файлы. Можно просто подключить готовые стили и скрипты в коде страницы.
- **Через пакетный менеджер:** Если ты пользуешься npm или yarn, можно установить его командой `npm install bootstrap`.

Как это выглядит в коде?

Если ты выберешь способ с CDN, тебе нужно будет вставить в шапку сайта (внутри тега `<head>`) вот такую ссылку:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/css/bootstrap.min.css" rel="stylesheet"
      integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
```

А перед закрывающимся тегом `</body>` вставить скрипты:

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/js/bootstrap.bundle.min.js"
      integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
```

Примечание: Версии могут меняться, лучше всегда брать актуальные ссылки на официальном сайте.

Система разметки Bootstrap

1. Контейнеры

В Bootstrap всё начинается с контейнера. Это базовый строительный блок, который содержит, выравнивает и центрирует содержимое страницы. Контейнеры бывают трех типов: фиксированный (.container), на всю ширину (.container-fluid) и адаптивные контейнеры (.container-sm, .container-md и т.д.).

```
<div class="container">
  <!-- Содержимое с отступами по бокам -->
</div>

<div class="container-fluid">
  <!-- Контейнер на всю ширину экрана -->
</div>

<div class="container-lg">
  <!-- Адаптивный контейнер (будет на всю ширину только на маленьких экранах) -->
</div>
```

2. Сетка

Bootstrap использует систему «12 колонок», 5 адаптивных ярусов, препроцессоров Sass и предустановленных классов.

Очень страшные слова, поэтому разберем на небольшом примере:

Представь, что ширина экрана разделена на 12 равных частей.

```
<div class="row">
  <div class="col-4">Заняли 4 колонки</div>
  <div class="col-8">Заняли 8 колонок</div>
</div>
```

Почему именно 12? Число 12 удобно тем, что делится на много частей: на 2, 3, 4, 6. Это дает гибкость в верстке. Например, можно создать макет из двух колонок по 6, трех колонок по 4, четырех колонок по 3 или шести колонок по 2.

Адаптивные ярусы (брейкпоинты)

Адаптивные ярусы (брейкпоинты) — это точки перелома, при которых макет страницы меняется в зависимости от ширины экрана. Bootstrap имеет 6 predetermined брейкпоинтов, основанных на размерах популярных устройств.

Ярус	Класс	Когда срабатывает	Какие устройства
Очень маленький	xs	<576px	Старые телефоны
Маленький	sm	≥576px	Современные телефоны
Средний	md	≥768px	Планшеты
Большой	lg	≥992px	Ноутбуки
Очень большой	xl	≥1200px	Большие мониторы

Пример адаптивной сетки:

Блок 1

Блок 2

Блок 3

```
<!-- 12 колонок на мобильном, 6 на планшете (md), 4 на ноутбуке (lg) -->
<div class="row">
  <div class="col-12 col-md-6 col-lg-4">Блок 1</div>
  <div class="col-12 col-md-6 col-lg-4">Блок 2</div>
  <div class="col-12 col-lg-4">Блок 3</div>
</div>
```

3. Колонки

Поскольку сетка Bootstrap построена на flexbox, мы можем использовать различные утилиты для выравнивания колонок по вертикали и горизонтали.

Вертикальное выравнивание для всех колонок в ряду

Классы `.align-items-*` на родителе `.row` выравнивают все колонки одновременно:

Верх (align-items-start)	Колонка 2	Колонка 3
Центр (align-items-center)	Колонка 2	Колонка 3
Выравнивание по нижнему краю (align-items-end)	Колонка 2	Колонка 3

```
<div class="row align-items-start">
  <div class="col">Вепх (align-items-start)</div>
  <div class="col">Колонка 2</div>
  <div class="col">Колонка 3</div>
</div>
<div class="row align-items-center">
  <div class="col">Центр (align-items-center)</div>
  <div class="col">Колонка 2</div>
  <div class="col">Колонка 3</div>
</div>
<div class="row align-items-end">
  <div class="col">Выравнивание по нижнему краю (align-items-end)</div>
  <div class="col">Колонка 2</div>
  <div class="col">Колонка 3</div>
</div>
```

Также можно выровнять каждую колонку индивидуально с помощью классов `.align-self-*`:

`.align-self-start`

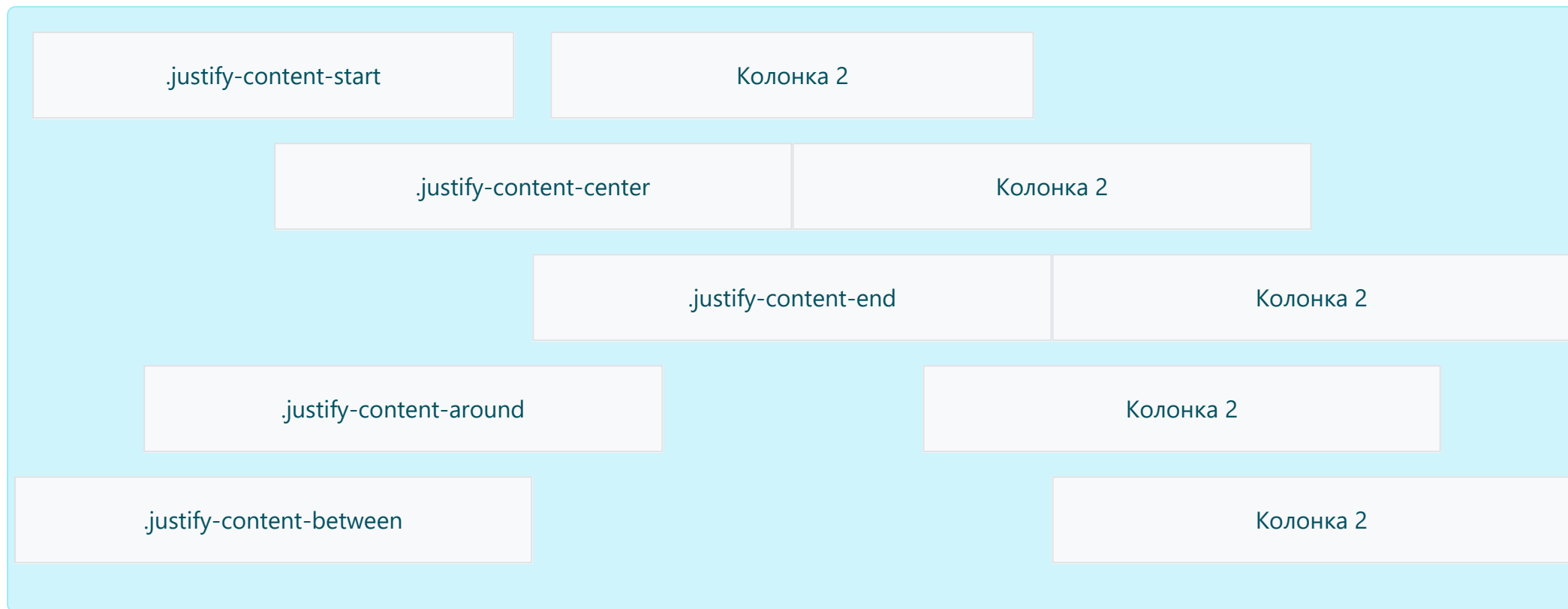
`.align-self-center`

`.align-self-end`

```
<div class="row">
  <div class="col align-self-start">...</div>
  <div class="col align-self-center">...</div>
  <div class="col align-self-end">...</div>
</div>
```

Горизонтальное выравнивание

Классы `.justify-content-*` на ряду управляют расположением колонок по горизонтали:



```
<div class="row justify-content-start">
  <div class="col-4">...</div>
  <div class="col-4">...</div>
</div>

<div class="row justify-content-center mb-3">
  <div class="col-4">...</div>
  <div class="col-4">...</div>
</div>
```

Обертывание колонок

Если сумма колонок в ряду превышает 12, лишние колонки автоматически переносятся на новую строку:

.col-9

.col-4

Поскольку $9 + 4 = 13 > 12$, этот div шириной в 4 колонки переносится на новую строку как один непрерывный блок.

.col-6

Последующие колонки продолжаютсся вдоль новой строки.

```
<div class="row">
  <div class="col-9">.col-9</div>
  <div class="col-4">.col-4 (переносится)</div>
  <div class="col-6">.col-6</div>
</div>
```

Разрывы колонок

Пустой `<div class="w-100"></div>` создаёт принудительный перенос на новую строку в нужном месте:

.col-6 .col-sm-4
4

.col-6 .col-sm-4
4

.col-6 .col-sm-4
4

.col-6 .col-sm-4
4

```
<div class="row">
  <div class="col-6 col-sm-4">1</div>
  <div class="col-6 col-sm-4">2</div>
  <div class="w-100"></div>
  <div class="col-6 col-sm-4">3</div>
  <div class="col-6 col-sm-4">4</div>
</div>
```

Комбинация с утилитами отображения позволяет делать разрыв только на определённых устройствах:

```
<!-- Разрыв только на md и выше -->
<div class="w-100 d-none d-md-block"></div>
```

Изменение порядка колонок

Классы **.order-*** меняют визуальный порядок колонок независимо от их порядка в HTML (доступны значения от 1 до 5):

Второй в HTML, но первый по порядку
(.order-1)

Третий в HTML, но второй по порядку
(.order-2)

Первый в HTML, но третий по порядку
(.order-3)

```
<div class="row">
  <div class="col order-3">Первый в HTML, но третий</div>
  <div class="col order-1">Второй в HTML, но первый</div>
  <div class="col order-2">Третий в HTML, но второй</div>
</div>
```

Классы **.order-first** и **.order-last** помещают колонку соответственно в начало или конец:

Третий в HTML, но первый (.order-first)

Второй в HTML, без
порядка

Первый в HTML, но последний (.order-last)

```
<div class="row">
  <div class="col order-last">Первый в HTML, но последний</div>
  <div class="col">Второй в HTML</div>
  <div class="col order-first">Третий в HTML, но первый</div>
</div>
```


Классы смещения .offset-*

Классы `.offset-*` увеличивают левый отступ колонки на указанное количество колонок:

`.col-md-4`

`.col-md-4 .offset-md-4`

`.col-md-3 .offset-md-3`

`.col-md-3 .offset-md-3`

`.col-md-6 .offset-md-3`

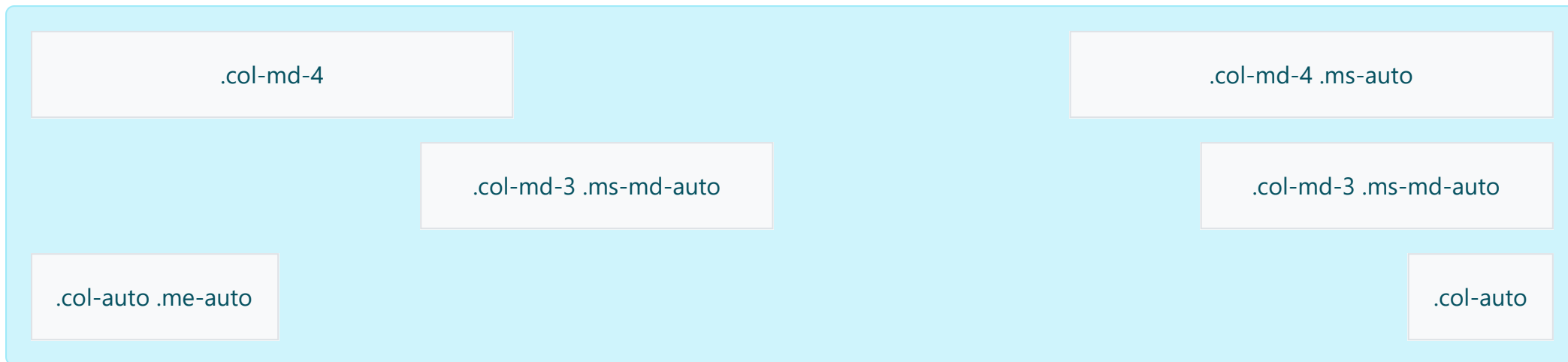
```
<div class="row">
  <div class="col-md-4">.col-md-4</div>
  <div class="col-md-4 offset-md-4">.col-md-4 .offset-md-4</div>
</div>
```

```
<div class="row">
  <div class="col-md-3 offset-md-3">.col-md-3 .offset-md-3</div>
  <div class="col-md-3 offset-md-3">.col-md-3 .offset-md-3</div>
</div>
```

```
<div class="row">
  <div class="col-md-6 offset-md-3">.col-md-6 .offset-md-3</div>
</div>
```

Утилиты полей (margin)

Утилиты `.ms-auto` (`margin-left: auto`) и `.me-auto` (`margin-right: auto`) позволяют прижимать колонки к краям или создавать переменные отступы:



```
<div class="row">
  <div class="col-md-4">.col-md-4</div>
  <div class="col-md-4 ms-auto">.col-md-4 .ms-auto</div>
</div>
<div class="row">
  <div class="col-md-3 ms-md-auto">.col-md-3 .ms-md-auto</div>
  <div class="col-md-3 ms-md-auto">.col-md-3 .ms-md-auto</div>
</div>
<div class="row">
  <div class="col-auto me-auto">.col-auto .me-auto</div>
  <div class="col-auto">.col-auto</div>
</div>
```

Автономные классы колонок

Классы `.col-*` можно использовать вне `.row` для задания ширины любому элементу — в таком случае отступы (gutter) не применяются:

`.col-3`: ширина 25%

`.col-sm-9`: ширина 75% на sm и выше

```
<div class="col-3 p-2">.col-3: ширина 25%</div>
<div class="col-sm-9 p-3">.col-sm-9: ширина 75% на sm и выше</div>
```

Это удобно для создания плавающих элементов, например изображений с обтеканием текстом:

Абзац текста, который обтекает изображение. Класс `.clearfix` нужен для правильной очистки обтекания. Изображение имеет классы `.col-md-6` (ширина 50% на md и выше), `.float-md-end` (обтекание справа) и отступы для красоты.

Такой подход позволяет создавать адаптивные макеты с обтеканием изображений текстом.

Изображение

```
<div class="clearfix">
  
  <p>Текст, обтекающий изображение...</p>
</div>
```

4. Отступы

Отступы между колонками используются для гибкого размещения и выравнивания содержимого в системе сеток Bootstrap. Они создаются с помощью `padding` у колонок и отрицательных `margin` у рядов.

Как это работает

- Горизонтальные отступы между столбцами создаются с помощью `padding`. Мы устанавливаем `padding-right` и `padding-left` для каждого столбца и используем отрицательное значение `margin` для смещения в начале и конце каждой строки, чтобы выровнять содержимое.
- По умолчанию отступ имеет ширину `1.5rem` (`24px`). Это позволяет сопоставить сетку с масштабом отступов и интервалов полей.
- Отступы могут быть быстро отрегулированы с помощью специальных классов.

Горизонтальные отступы (`.gx-*`)

Классы `.gx-*` управляют шириной горизонтальных промежутков между колонками. Доступны значения от 0 до 5:

Колонка с отступом

Колонка с отступом

```
<div class="container px-4 text-center">
  <div class="row gx-5">
    <div class="col"><div class="p-3">Содержание столбца</div></div>
    <div class="col"><div class="p-3">Содержание столбца</div></div>
  </div>
</div>
```

`.gx-5` создает большие отступы между колонками. При использовании больших отступов может потребоваться корректировка контейнера с помощью утилит `padding`, например `.px-4`.

Вертикальные отступы (.gy-*)

Классы **.gy-*** управляют вертикальными промежутками между строками колонок:

Колонка 1

Колонка 2

Колонка 3

Колонка 4

```
<div class="container text-center">
  <div class="row gy-5">
    <div class="col-6">Колонка 1</div>
    <div class="col-6">Колонка 2</div>
    <div class="col-6">Колонка 3</div>
    <div class="col-6">Колонка 4</div>
  </div>
</div>
```

Горизонтальные и вертикальные отступы (.g-*)

Классы **.g-*** одновременно задают и горизонтальные, и вертикальные отступы:

Колонка с отступами со всех сторон

Колонка с отступами со всех сторон

Колонка с отступами со всех сторон

Колонка с отступами со всех сторон

```
<div class="container text-center">
  <div class="row g-4">
    <div class="col-6">Колонка</div>
    <div class="col-6">Колонка</div>
    <div class="col-6">Колонка</div>
    <div class="col-6">Колонка</div>
  </div>
</div>
```

Без промежутков между колонками (.g-0)

Класс **.g-0** полностью удаляет отступы между колонками. Он убирает отрицательные margin у **.row** и горизонтальные padding у всех дочерних колонок:

.col-sm-6 .col-md-8

.col-6 .col-md-4

```
<div class="row g-0 text-center">
  <div class="col-sm-6 col-md-8">.col-sm-6 .col-md-8</div>
  <div class="col-6 col-md-4">.col-6 .col-md-4</div>
</div>
```

Это полезно для создания "сквозных" дизайнов, где элементы должны прилегать друг к другу вплотную.

Ряды столбцов с отступами

Классы отступов можно комбинировать с классами `.row-cols-*` для создания адаптивных сеток с управляемыми промежутками:

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

Колонка ряда

```
<div class="row row-cols-2 row-cols-lg-4 g-4">
  <div class="col">
    <div class="p-3">Колонка ряда</div>
  </div>
  <div class="col">
    <div class="p-3">Колонка ряда</div>
  </div>
  ...
</div>
```

Доступные значения отступов

Классы отступов строятся из карты Sass `$gutters`, которая унаследована от карты `$spacers`. Доступны значения от 0 до 5:

Класс	Значение	Размер
<code>.g-0</code> , <code>.gx-0</code> , <code>.gy-0</code>	0	0
<code>.g-1</code> , <code>.gx-1</code> , <code>.gy-1</code>	<code>\$spacer * 0.25</code>	0.25rem (4px)
<code>.g-2</code> , <code>.gx-2</code> , <code>.gy-2</code>	<code>\$spacer * 0.5</code>	0.5rem (8px)
<code>.g-3</code> , <code>.gx-3</code> , <code>.gy-3</code>	<code>\$spacer</code>	1rem (16px)
<code>.g-4</code> , <code>.gx-4</code> , <code>.gy-4</code>	<code>\$spacer * 1.5</code>	1.5rem (24px) - значение по умолчанию
<code>.g-5</code> , <code>.gx-5</code> , <code>.gy-5</code>	<code>\$spacer * 3</code>	3rem (48px)

Классы можно делать адаптивными, добавляя брейкпоинты: `.gx-md-4`, `.gy-lg-2`, `.g-sm-3` и т.д.

Автоматическая компоновка колонок

Равная ширина

Если не указывать цифру в классе `col`, все колонки будут одинаковой ширины:

Колонка 1

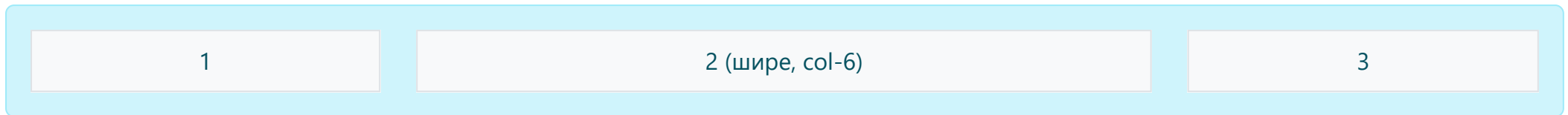
Колонка 2

Колонка 3


```
<div class="row">
  <div class="col">Колонка 1</div>
  <div class="col">Колонка 2</div>
  <div class="col">Колонка 3</div>
</div>
```

Установка ширины одной колонки

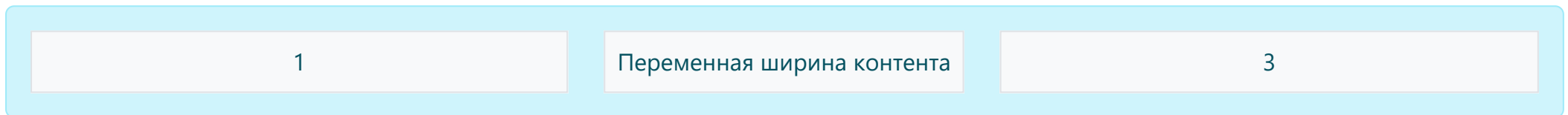
Можно задать ширину одной колонки, остальные подстроятся автоматически:



```
<div class="row">
  <div class="col">1</div>
  <div class="col-6">2</div>
  <div class="col">3</div>
</div>
```

Переменная ширина контента (col-auto)

Класс col-auto заставляет колонку занимать ровно столько места, сколько нужно её содержимому:



```
<div class="row">
  <div class="col col-lg-2">1</div>
  <div class="col-md-auto">Переменная ширина контента</div>
  <div class="col col-lg-2">3</div>
</div>
```

Пример использования адаптивности

Рассмотрим пример с маркетплейсом Yargy Market, где нужно разместить карточки товаров. Для каждого типа устройства требуется своя разметка, так как на экране телефона помещается меньше информации, чем на большом мониторе. Используя классы Bootstrap, мы можем задать разное количество колонок для разных устройств.

Товар №1

Товар №2

Товар №3

Товар №4

Визуализация расположения на разных устройствах:

Телефон

Товар №1

Товар №2

Товар №3

Товар №4

1 колонка (col-12)

Планшет

Товар №1

Товар №2

Товар №3

Товар №4

2 колонки (col-md-6)

Ноутбук

Товар №1

Товар №2

Товар №3

Товар №4

3 колонки (col-lg-4)

Монитор

Товар №1

Товар №2

Товар №3

Товар №4

4 колонки (col-xl-3)

Классы .row-cols-*

Классы .row-cols-* — это специальные классы Bootstrap, которые задают количество колонок в ряду прямо на родительском контейнере .row. Вместо того чтобы писать класс для каждой колонки отдельно, мы пишем один класс на весь ряд. Это особенно удобно при работе с динамическим контентом, когда количество элементов может меняться.

Синтаксис:

- `.row-cols-{количество}` — одинаковое количество колонок на всех экранах
- `.row-cols-{breakpoint}-{количество}` — адаптивное количество колонок
- `.row-cols-auto` — колонки подстраиваются под ширину содержимого

Пример с разным количеством на разных устройствах:

1
4
7

2
5
8

3
6

```
<div class="row row-cols-1 row-cols-sm-2 row-cols-md-3 row-cols-lg-6">  
  <div class="col">1</div>  
  <div class="col">2</div>  
  ...  
</div>
```

Пример с .row-cols-auto:

Короткий

Средний текст

Очень длинный текст для примера

Ещё

Каждая колонка занимает ровно столько места, сколько нужно её содержимому.

Sass (Syntactically Awesome Style Sheets)

Sass — это препроцессор CSS, который расширяет возможности обычного CSS. Он позволяет использовать переменные, вложенность, миксины и другие возможности, делая написание стилей более удобным и поддерживаемым. Bootstrap написан на Sass, поэтому его легко настраивать под свои нужды.

Основные возможности Sass:

- **Переменные** — можно задать значения (цвета, отступы) один раз и использовать их многократно
- **Вложенность** — позволяет писать стили в иерархическом виде, как в HTML
- **Миксины** — переиспользуемые блоки CSS-кода
- **Наследование** — один селектор может наследовать стили другого

Обычный CSS

```
.button {
  background-color: blue;
  color: white;
  padding: 10px;
}

.button-large {
  background-color: blue;
  color: white;
  padding: 20px;
}
```

Sass

```
$main-color: blue;
$text-color: white;

.button {
  background-color: $main-color;
  color: $text-color;
  padding: 10px;

  &-large {
    padding: 20px;
  }
}
```

В примере выше видно, как Sass позволяет избежать повторения кода. Мы создали переменные для цветов и использовали вложенность (&-large) для создания модификатора кнопки. Это делает код более чистым и легким в поддержке.

Переменные сетки в Sass

Bootstrap использует следующие переменные для настройки сетки:

```
// Количество колонок
$grid-columns: 12;

// Отступы между колонками (gutter)
$grid-gutter-width: 1.5rem;

// Максимальное количество для .row-cols-*
$grid-row-columns: 6;

// Контрольные точки (breakpoints)
$grid-breakpoints: (
  xs: 0,
  sm: 576px,
  md: 768px,
  lg: 992px,
  xl: 1200px,
  xxl: 1400px
);

// Ширина контейнеров
$container-max-widths: (
  sm: 540px,
  md: 720px,
  lg: 960px,
  xl: 1140px,
  xxl: 1320px
);
```

Миксины для создания своей сетки

```
// Создание контейнера
.my-custom-container {
  @include make-container();
}

// Создание ряда
.my-custom-row {
  @include make-row();
}

// Подготовка колонки
.my-custom-col {
  @include make-col-ready();
  @include make-col(6); // колонка на 6 частей
}

// Колонка со смещением
.my-col-offset-3 {
  @include make-col-offset(3);
}
```

Настройка сетки под себя

Вы можете изменить параметры сетки, переопределив переменные Sass до подключения Bootstrap:

```
// Например, сделать 10-колоночную сетку
$grid-columns: 10;

// Уменьшить отступы между колонками
$grid-gutter-width: 1rem;

// Добавить свою контрольную точку
$grid-breakpoints: (
  xs: 0,
  sm: 576px,
  md: 768px,
  lg: 992px,
  xl: 1200px,
  xxl: 1400px,
  xxxl: 1600px // новая точка
);

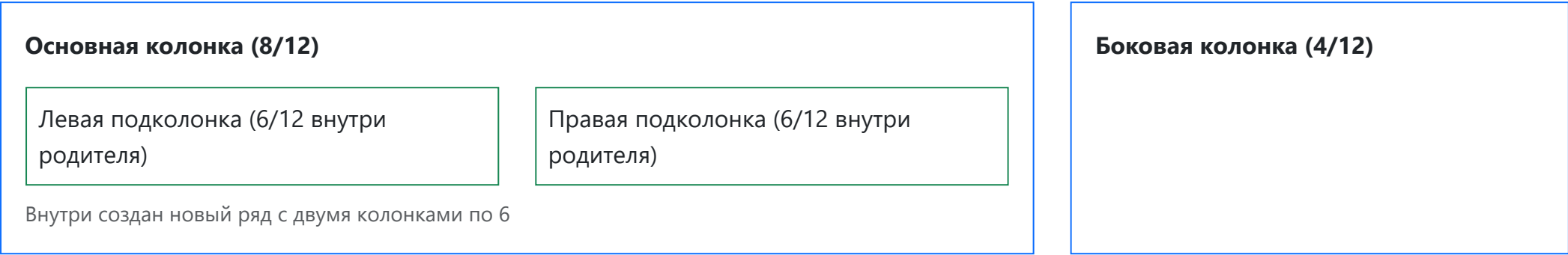
// Затем подключить Bootstrap
@import "bootstrap";
```

Примечание: После изменения переменных нужно перекомпилировать Sass в CSS.

Вложенность сетки

Вложенность — это возможность создавать новую сетку внутри существующей колонки. Это позволяет строить сложные макеты с дополнительной структурой внутри основных блоков. Например, внутри основной колонки с контентом можно создать еще один ряд с несколькими колонками.

Важное правило: Вложенные колонки считаются относительно родительской колонки, а не всей страницы. Если родитель занимает 8 колонок из 12, то внутри него колонка с классом col-6 будет занимать половину от этих 8 колонок (то есть 4 колонки от общей ширины страницы). Сумма вложенных колонок не должна превышать 12, но может быть меньше.



В данном примере основная колонка занимает 8 частей из 12. Внутри нее создан новый ряд с двумя колонками, каждая из которых занимает по 6 частей внутри родителя. Таким образом, левая и правая подколонки фактически занимают по 4 части от общей ширины страницы (половина от 8).

Что такое формы в Bootstrap?

Формы в Bootstrap — это набор стилей и компонентов для создания удобных, адаптивных и доступных форм. Они включают поля ввода, чекбоксы, радиокнопки, селекты, группы ввода и валидацию. Формы обеспечивают единый вид в разных браузерах и устройствах.

Bootstrap использует классы вроде `form-control` для стилизации элементов, делая их современными и отзывчивыми.

Обзор форм в Bootstrap

Формы строятся на классах вроде `form-control` для полей ввода, `form-label` для меток и `form-text` для подсказок. Например:

```
<form>
  <div class="mb-3">
    <label for="exampleInputEmail1" class="form-label">Адрес электронной почты</label>
    <input type="email" class="form-control" id="exampleInputEmail1" aria-describedby="emailHelp" />
    <div id="emailHelp" class="form-text">Мы никогда не поделимся вашей почтой.</div>
  </div>
  <button type="submit" class="btn btn-primary">Отправить</button>
</form>
```

Адрес электронной почты

Мы никогда не поделимся вашей почтой.

Отправить

1. Элементы управления формами (Form Controls)

Класс `form-control` применяется к `<input>`, `<textarea>` и `<select>`. Поддерживает типы: text, email, password, number, file и т.д. Размеры: `form-control-sm`, `form-control-lg`.

```
<input type="text" class="form-control" placeholder="Введите текст" />
<textarea class="form-control" rows="3"></textarea>
```

Стандартный текстовый ввод

Маленький численный ввод (`.form-control-sm`)

Большой парольный ввод (`.form-control-lg`)

2. Выпадающие списки (Select)

Класс `form-select` для `<select>`. Поддерживает multiple, size, disabled. Размеры: `form-select-sm`, `form-select-lg`.

```
<select class="form-select">
  <option selected>Выберите...</option>
  <option value="1">Один</option>
</select>
```

Выберите...

Отключенный селект

3. Чекбоксы и радиокнопки (Checks and Radios)

Класс `form-check` для чекбоксов и радиокнопок. Для свитчей — `form-switch`. Inline: `form-check-inline`. Без меток: используйте `aria-label`.

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="check1" />
  <label class="form-check-label" for="check1">Стандартный чекбокс</label>
</div>
```

☐ Стандартный чекбокс

Варианты расположения и стилизации

Чекбоксы и радиокнопки можно располагать горизонтально, вертикально, делать кнопочного вида и использовать свитчи.

Горизонтальное (Inline) расположение

☐ 1 ☐ 2 ☐ 3 (отключено)

☐ 1 ☐ 2 ☐ 3 (отключенный)

☒ Свитч

Кнопочного вида

Класс `btn-check` + `btn` превращает чекбоксы и радиокнопки в красивые кнопки-переключатели.

Чекбокс 1 Чекбокс 2 Чекбокс 3 (выбран) Вкл Выкл

4. Ползунки (Range)

Класс `form-range` для `<input type="range">`. Поддерживает `min`, `max`, `step`, `disabled`.

```
<input type="range" class="form-range" min="0" max="5" step="0.5" id="customRange" />
```



5. Группы ввода (Input Group)

Группы ввода позволяют добавлять текст, иконки, кнопки, выпадающие списки слева и/или справа от поля ввода. Классы: `input-group`, `input-group-text`.

```
<div class="input-group">
  <span class="input-group-text">Область текста</span>
  <textarea class="form-control" aria-label="With textarea"></textarea>
</div>
```

Область текста

Базовые примеры

@Username

Username получателя

@example.com

\$

.00

С кнопками

Кнопка

Username получателя

Кнопка

Выпадающее ▾

С чекбоксами / радиокнопками

☒

☐

Несколько дополнений

С

Двумя

дополнениями

6. Плавающие метки (Floating Labels)

Класс `form-floating` для плавающих меток над полями. **Требует placeholder.**

```
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="floatingInput" placeholder="">
  <label for="floatingInput">Плавающая метка</label>
</div>
```

Плавающая метка

Плавающая метка

Метка уходит вверх, освобождая место для основного текста

Плавающие метки с группами ввода

@ Email address

Плавающие метки в сетке

При работе с системой сеток Bootstrap обязательно размещайте элементы формы в классах столбцов.

Email address

Работает с selects
Открыть меню



7. Макеты форм (Layout) — подробный обзор

По умолчанию элементы форм (`input`, `select`, `textarea`) занимают 100% ширины родителя и располагаются вертикально. Для создания многостолбцовых, горизонтальных или адаптивных форм используется система сетки Bootstrap.

Сетка форм

Оборачивайте поля в `.row` и используйте `.col-*`. Горизонтальные отступы регулируются классами `.g-*` (gutters).

```
<div class="bd-example">
  <div class="row g-3">
    <div class="col-md-6">
      <input type="text" class="form-control" placeholder="Имя" />
    </div>
    <div class="col-md-6">
      <input type="text" class="form-control" placeholder="Фамилия" />
    </div>
    <div class="col-12">
      <input type="email" class="form-control" placeholder="Email" />
    </div>
  </div>
</div>
```

Горизонтальные формы

Используйте класс `.col-form-label` для выравнивания меток по вертикали с полями.

Email

Пароль

Авторазмер колонок (.col-auto)

Колонка занимает ровно столько места, сколько нужно содержимому. Удобно для форм поиска, фильтров, логинов.

Имя пользователя

@ Ник

Отправить

Адаптивные строчные формы (row-cols-auto)

На больших экранах элементы выстраиваются в ряд по горизонтали, на мобильных — стакаются по вертикали.

@ Username

Выберите роль

Войти

8. Валидация форм (Validation)

Bootstrap использует встроенную HTML5-валидацию + стили для состояний `:valid` / `:invalid`. Есть два основных подхода: клиентская (браузерная) и серверная.

Клиентская валидация (рекомендуемый способ)

Самый удобный вариант для пользователя: он ещё не отправил форму, а уже видит «Эй, тут забыли email» или «Пароль слишком короткий».

Что нужно сделать:

1. Добавить на форму `class="needs-validation"` и атрибут `novalidate` (это отключает стандартные всплывающие окошки браузера).
2. На обязательные поля навесить атрибут `required`, тип `type="email"`, `minlength` и т.д. — браузер сам проверит.
3. Под каждым полем добавить блок с сообщением об ошибке: `<div class="invalid-feedback">Текст ошибки</div>`
4. Немного JavaScript, чтобы при отправке формы добавлялся класс `was-validated` — именно он включает красивые стили Bootstrap.

```
<div class="bd-example">
  <form class="needs-validation" novalidate>
    <div class="mb-3">
      <label for="userEmail" class="form-label">Email</label>
      <input type="email" class="form-control" id="userEmail" required />
      <div class="invalid-feedback">Введите корректный email</div>
    </div>
    <div class="mb-3">
      <label for="userName" class="form-label">Имя</label>
      <input type="text" class="form-control" id="userName" required minlength="2" />
      <div class="invalid-feedback">Имя должно быть хотя бы из 2 букв</div>
    </div>
    <button class="btn btn-primary" type="submit">Отправить</button>
  </form>
</div>
```

Email

Имя

Отправить

```
<!-- JavaScript (обычно в конце страницы или в отдельном файле) -->
<script>
(() => {
  'use strict'
  const form = document.querySelector('.needs-validation')
  form.addEventListener('submit', event => {
    if (!form.checkValidity()) {
      event.preventDefault()
      event.stopPropagation()
    }
    form.classList.add('was-validated')
  }, false)
})()
</script>
```

После этого поле с ошибкой получит красную рамку, под ним появится сообщение, а если всё ок — зелёная рамка и галочка.

Email

A@AAA.AA



Имя



Имя должно быть хотя бы из 2 букв

Отправить

Серверная валидация

Если форма уже улетела на сервер, а там выяснилось, что логин занят или код неверный — сервер просто возвращает страницу с подсвеченными полями.

Как сделать:

- К полю добавляем класс `is-invalid` (ошибка) или `is-valid` (всё хорошо).
- Под полем — тот же `invalid-feedback` или `valid-feedback`.
- Если поле внутри `input-group` — добавить класс `has-validation` на группу, иначе рамка может криво отображаться.

```
<div class="bd-example">
  <div class="mb-3">
    <label for="login" class="form-label">Логин</label>
    <input type="text" class="form-control is-invalid" id="login" value="admin" />
    <div class="invalid-feedback">Такой логин уже занят, придумайте другой</div>
  </div>
</div>
```

Логин



Такой логин уже занят, придумайте другой

Где показывать сообщения: под полем или всплывашкой?

- **Обычный текст под полем** — классы `invalid-feedback` / `valid-feedback`.
- **Всплывающая подсказка (tooltip)** — классы `invalid-tooltip` / `valid-tooltip`. Нужно обернуть поле в `<div class="position-relative">`, подсказка появляется при наведении или фокусе. Используется в случаях, когда места мало.



Ник занят!

9. Доступность и отключенные состояния

Bootstrap делает формы максимально доступными, но требует правильного использования атрибутов.

Отключенные поля и группы (disabled / readonly)

Есть три основных состояния:

- **disabled** — поле полностью неактивно, не отправляется на сервер, выглядит серым и бледным
- **readonly** — поле можно выделять и копировать, но нельзя редактировать, отправляется на сервер
- **<fieldset disabled>** — отключает сразу всю группу полей

```
<div class="bd-example">
  <fieldset disabled>
    <legend>Отключенная группа</legend>
    <div class="mb-3">
      <label class="form-label">Имя</label>
      <input type="text" class="form-control" placeholder="Недоступно" />
    </div>
    <div class="form-check">
      <input class="form-check-input" type="checkbox" />
      <label class="form-check-label">Нельзя отметить</label>
    </div>
    <button type="submit" class="btn btn-primary">Отправить</button>
  </fieldset>
</div>
```

Отключенная группа

Имя

Недоступно

☐ Нельзя отметить

Отправить

Доступность: метки и ARIA

ARIA (Accessible Rich Internet Applications) — это набор специальных HTML-атрибутов, которые помогают сделать веб-интерфейсы понятными для скринридеров и других вспомогательных технологий. Они не меняют внешний вид страницы, но сообщают программам чтения с экрана, что именно происходит: где метка, где ошибка, какое поле отключено и т.д.

- Каждое поле ввода должно иметь понятную метку — лучше всего через `<label for="id">`. Если видимой метки нет (например, только иконка), используйте `aria-label` или `aria-labelledby`.
- Подсказки, сообщения об ошибках и пояснения под полем связывайте с помощью `aria-describedby="id-подсказки"`. Тогда скринридеры сразу озвучат важную информацию после поля.
- Для отключённых кнопок и элементов (кроме обычного `disabled`) добавляйте: `aria-disabled="true"` и `tabindex="-1"` — чтобы их не фокусировали клавиатурой.

`<!-- Пример: поле + подсказка + ошибка -->`

```
<input class="form-control is-invalid" id="email" aria-describedby="emailHelp emailFeedback">
```

```
<div id="emailHelp" class="form-text">Мы не передаём почту третьим лицам</div>
```

```
<div id="emailFeedback" class="invalid-feedback">Введите корректный email</div>
```

10. Настройка форм через Sass

Bootstrap написан на Sass, поэтому почти все параметры форм (отступы, цвета границ, размер шрифта, толщину фокусной обводки и т.д.) можно переопределить, не трогая основной код.

Самые часто меняемые переменные форм

Переменная	Значение по умолчанию	За что отвечает
<code>\$input-padding-y</code>	<code>.375rem</code>	Вертикальный внутренний отступ поля
<code>\$input-padding-x</code>	<code>.75rem</code>	Горизонтальный внутренний отступ
<code>\$input-font-size</code>	<code>1rem</code>	Размер текста внутри поля
<code>\$input-border-radius</code>	<code>.375rem</code>	Скругление углов
<code>\$input-focus-width</code>	<code>.25rem</code>	Толщина синей обводки при фокусе
<code>\$form-text-color</code>	<code>#6c757d</code>	Цвет мелкого текста-подсказки

Простой пример кастомизации

```
// Перед импортом Bootstrap переопределяем нужные переменные
$input-padding-y: .5rem;           // чуть выше поля
$input-padding-x: 1rem;            // шире поля
$input-font-size: 1.05rem;         // чуть крупнее текст
$input-focus-width: .3rem;         // заметнее фокус
$input-focus-border-color: #6f42c1; // другой цвет обводки при фокусе

// Можно также поменять цвета валидации
$form-valid-color: #198754;
$form-invalid-color: #dc3545;

// Затем подключаем
@import "bootstrap/scss/bootstrap";
```

Утилиты Bootstrap

Утилиты в Bootstrap — это набор готовых классов, которые позволяют изменить или дополнить существующий стиль элементов на странице. Они предназначены для быстрой стилизации без написания кастомного CSS.

Подробнее о них можно прочитать в [документации](#).

Служебный API

Служебный API - это инструмент на основе Sass для создания служебных классов. Утилиты Bootstrap создаются с помощью служебного API и могут использоваться для изменения или расширения нашего набора служебных классов по умолчанию через Sass.

Все служебные переменные добавляются в переменную `$utilities` в нашей таблице стилей `_utilities.scss`.

- **Свойство:** Требуемый ключ `property` должен быть установлен для любой утилиты, и он должен содержать допустимое свойство CSS. Это свойство используется в сгенерированном наборе правил утилиты. Когда ключ `class` опущен, он также служит именем класса по умолчанию.
- **Значения:** Используйте ключ `values`, чтобы указать, какие значения для указанного `property` должны использоваться в сгенерированных именах классов и правилах. Может быть списком или картой (задается в утилитах или в переменной Sass).
- **Класс:** Используйте опцию `class`, чтобы изменить префикс класса, используемый в скомпилированном CSS.
- **Утилиты переменных CSS:** Установите логическую опцию `css-var` в значение `true`, и API будет генерировать локальные переменные CSS для данного селектора вместо обычных правил `property: value`. Добавьте необязательное `css-variable-name`, чтобы задать имя переменной CSS, отличное от имени класса.
- **Локальные переменные CSS:** Используйте опцию `local-vars`, чтобы указать карту Sass, которая будет генерировать локальные переменные CSS в наборе правил служебного класса. Обратите внимание, что для использования этих локальных переменных CSS в сгенерированных правилах CSS может потребоваться дополнительная работа.
- **Состояния:** Используйте опцию `state` для генерации вариаций псевдокласса. Примеры псевдоклассов: `:hover` и `:focus`. Когда предоставляется список состояний, для этого псевдокласса создаются имена классов. Например, чтобы изменить непрозрачность при наведении указателя мыши, добавьте `state: hover`, и Вы получите `.opacity-hover: hover` в Вашем скомпилированном CSS.
- **Адаптивность:** Добавьте логическое значение `responsive` для создания адаптивных утилит для всех контрольных точек.
- **Печать:** Включение опции `print` также сгенерирует служебные классы для печати, которые применяются только в медиа-запросе `@media print {...}`.

- **Важность:** Все утилиты, генерируемые API, включают `!important`, чтобы гарантировать, что они переопределяют компоненты и классы модификаторов должным образом. Вы можете переключать этот параметр глобально с помощью переменной `$enable-important-utilities` (по умолчанию `true`).
- **Использование API:** Теперь, когда Вы знакомы с тем, как работает API утилит, узнайте, как добавлять свои собственные классы и изменять наши утилиты по умолчанию.

Использование API.

- **Переопределение утилит:** Переопределите существующие утилиты, используя тот же ключ.
- **Добавление утилит:** Новые утилиты могут быть добавлены к карте по умолчанию `$utilities` с помощью `map-merge`. Убедитесь, что наши необходимые файлы Sass и `_utilities.scss` сначала импортированы, а затем используйте `map-merge`, чтобы добавить Ваши дополнительные утилиты.
- **Изменение утилит:** Измените существующие утилиты в карте по умолчанию `$utilities` с помощью функций `map-get` и `map-merge`. В приведенном ниже примере мы добавляем дополнительное значение к утилитам `width`. Начните с начального `map-merge`, а затем укажите, какую утилиту Вы хотите изменить. Оттуда извлеките вложенную карту `"width"` «`map-get`» для доступа и изменения параметров и значений утилиты.
- **Включение адаптивности:** Вы можете включить адаптивные классы для существующего набора утилит, которые в настоящее время не отвечают по умолчанию.
- **Переименование утилит:** API утилит можно использовать для переопределения результирующего `class` данной утилиты.
- **Удаление утилит:** Удалите все стандартные утилиты с помощью Sass-функции `map-remove()`.

Фон

Передайте смысл с помощью background-color и добавьте украшения с помощью градиентов.

- Фоновый цвет:**
Подобно классам цвета контекстного текста, установите фон элемента для любого контекстного класса. Фоновые утилиты не устанавливают color.
- Фоновый градиент:**
При добавлении класса .bg-gradient, линейный градиент добавляется как фоновое изображение к фону. Этот градиент начинается с полупрозрачного белого цвета, который постепенно исчезает к низу.
Если нужен градиент в вашем собственном CSS, просто добавьте background-image: var(--bs-gradient);
- Непрозрачность:**
Начиная с версии 5.1.0, утилиты background-color генерируются с помощью Sass с использованием переменных CSS. Это позволяет изменять цвет в реальном времени без компиляции и динамических изменений альфа-прозрачности.
Чтобы изменить непрозрачность, переопределите --bs-bg-opacity с помощью пользовательских стилей или встроенных стилей. Или выберите любую из утилит .bg-opacity.

.bg-primary	.bg-primary-subtle	.bg-secondary	.bg-secondary-subtle	.bg-success
.bg-success-subtle	.bg-danger	.bg-danger-subtle	.bg-warning	.bg-warning-subtle
.bg-info	.bg-info-subtle	.bg-light	.bg-light-subtle	.bg-dark
.bg-dark-subtle	.bg-body-secondary	.bg-body-tertiary	.bg-body	.bg-black
.bg-white	.bg-transparent	Успех	Успех 50%	Успех 10%

Границы

Используйте утилиты границ, чтобы быстро создать границу и граничный радиус элемента. Отлично подходит для изображений, кнопок или любого другого элемента.

Используйте служебные программы для создания границ, чтобы добавить или удалить границы элемента. Выбирайте из всех границ или по одной.

- **Добавление:** Добавьте границы к пользовательским элементам
- **Вычитание:** Или удалите границы
- **Цвет:** Измените цвет границы с помощью утилит, построенных на цветах нашей темы. Или измените `border-color` компонента по умолчанию.
- **Непрозрачность:** Аналогично непрозрачности фона, можно изменить с помощью переопределения `--bs-border-opacity` или выбрать любую из утилит `.border-opacity`.
- **Ширина:** Используя класс `border-1`, можно изменить ширину границ. Доступно 5 вариантов ширины.
- **Радиус:** Добавьте классы к элементу, чтобы легко изменить его углы. Например, `class="rounded-start"` или `class="rounded-circle"`.
- **Размеры:** Используйте классы масштабирования для больших или меньших закругленных углов. Размеры варьируются от 0 до 5 и могут быть настроены путем изменения API утилит.

Адрес электронной почты

name@example.com

Опасный заголовок

Изменение цвета и ширины границы

Цвета

Передайте смысл через color с помощью нескольких служебных классов цвета. Также включает поддержку стилизации ссылок с помощью состояний наведения.

Непрозрачность: Чтобы изменить эту непрозрачность, переопределите `--bs-text-opacity` с помощью пользовательских стилей или встроенных стилей. Или выберите любую из утилит `.text-opacity`.

<code>.text-primary</code>	<code>.text-primary-emphasis</code>	<code>.text-secondary</code>	<code>.text-secondary-emphasis</code>	<code>.text-success</code>
<code>.text-success-emphasis</code>	<code>.text-danger</code>	<code>.text-danger-emphasis</code>	<code>.text-warning</code>	<code>.text-warning-emphasis</code>
<code>.text-info</code>	<code>.text-info-emphasis</code>	<code>.text-light</code>	<code>.text-light-emphasis</code>	<code>.text-dark</code>
<code>.text-dark-emphasis</code>	<code>.text-body</code>	<code>.text-body-emphasis</code>	<code>.text-body-secondary</code>	<code>.text-body-tertiary</code>
<code>.text-black</code>	<code>.text-white</code>	<code>.text-black-50</code>	<code>.text-white-50</code>	Основной текст
Основной 50%	Основной 25%			

Отображение элементов

Быстрое переключение отображаемых элементов и многого другого с помощью утилит отображения. Включает поддержку некоторых из наиболее распространенных значений, а также некоторые дополнительные функции для управления отображением при печати.

- **Обозначение:** Утилиты отображения классов, которые применяются ко всем контрольным точкам, от xs до xxl, не имеют в себе аббревиатуры контрольной точки. Это потому, что эти классы применяются начиная с `min-width: 0;` и выше, и поэтому не связаны медиа-запросами. Остальные контрольные точки, однако, содержат аббревиатуру контрольной точки. Таким образом, классы именуются в следующем формате:

- `.d-{value}` для xs
- `.d-{breakpoint}-{value}` для sm, md, lg, xl и xxl

Где значение - одно из: none, inline, inline-block, block, grid, inline-grid, table, table-cell, table-row, flex, inline-flex.

- **Скрытие элементов:** Для более быстрой разработки, удобной для мобильных устройств, используйте адаптивные классы отображения для отображения и скрытия элементов по устройствам. Избегайте создания совершенно разных версий одного и того же сайта, вместо этого скройте элементы в зависимости от размера экрана. Чтобы скрыть элементы, просто используйте класс `.d-none` или один из классов `.d-{sm,md,lg,xl,xxl}-none` для любого варианта адаптивного экрана. Чтобы отображать элемент только в заданном интервале размеров экрана, Вы можете объединить один класс `.d-*-none` с классом `.d-*-{value}`, например, `.d-none .d-md-block .d-xl-none .d-xxl-none` скроет элемент для всех размеров экрана, кроме средних и больших устройств.
- **Отображение при печати:** Измените значение `display` элементов при печати с помощью служебных классов отображения при печати. Включает поддержку тех же значений `display`, что и наши адаптивные утилиты `.d-*`. Классы печати и отображения можно комбинировать.

d-inline

d-inline

Flex

Быстрое управление макетом, выравниванием и размером столбцов сетки, навигацией, компонентами и многим другим с помощью полного набора адаптивных утилит flexbox. Для более сложных реализаций может потребоваться пользовательский CSS.

- **Включение «гибкого» поведения:** Применяйте утилиты display для создания гибкого контейнера и трансформируйте прямые дочерние элементы в «гибкие». Гибкие контейнеры и элементы можно изменять дополнительными свойствами гибкости. Для этого можно использовать класс d-flex или d-inline-flex.
- **Направление:** Задайте направление гибких элементов в гибком контейнере с помощью утилит направления. В большинстве случаев Вы можете опустить здесь горизонтальный класс, так как браузер по умолчанию равен row. Однако Вы можете столкнуться с ситуациями, когда Вам нужно явно установить это значение (например, адаптивные макеты). Используйте .flex-row, чтобы установить горизонтальное направление (по умолчанию в браузере), или .flex-row-reverse, чтобы начать горизонтальное направление с противоположной стороны. Используйте .flex-column, чтобы задать вертикальное направление, или .flex-column-reverse, чтобы задать вертикальное направление с противоположной стороны.
- **Выравнивание содержимого:** Используйте утилиты justify-content в контейнерах flexbox, чтобы изменить выравнивание элементов Флекс по главной оси (ось x для начала, ось y, если flex-direction: column). Выберите start (по умолчанию в браузере), end, center, between, around или evenly.
- **Выравнивание элементов:** Используйте утилиты align-items на контейнерах flexbox, чтобы изменить выравнивание элементов Флекс по поперечной оси (ось Y в начале, ось X, если flex-direction: column). Выберите start, end, center, baseline или stretch (по умолчанию в браузере).
- **Сплошное выравнивание:** Используйте утилиты align-self для элементов flexbox, чтобы индивидуально изменить их выравнивание по поперечной оси (ось Y для начала, ось X, если flex-direction: column). Выберите один из тех же параметров, что и flex-direction: column). Выберите один из тех же вариантов, что и align-items: start, end, center, baseline или stretch (по умолчанию в браузере).

- **Заполнение:** Используйте класс `.flex-fill` для серии родственных элементов, чтобы заставить их ширину равную их содержимому (или равную ширину, если их содержимое не превышает их границы), занимая при этом все доступное горизонтальное пространство.
- **Увеличение и уменьшение:** Используйте утилиты `.flex-grow-*` для переключения способности элемента Флекс расти, чтобы заполнить доступное пространство. В приведенном ниже примере элементы `.flex-grow-1` используют все доступное пространство, которое могут, а оставшимся двум элементам Флекс их необходимое пространство.
- **Автоматические поля:** Flexbox может делать довольно удивительные вещи, когда Вы смешиваете выравнивание гибкости с автоматическими полями.
- **С элементами выравнивания:** Вертикально переместите один элемент Флекс вверх или вниз контейнера, смешав `align-items`, `flex-direction: column` и `margin-top: auto` или `margin-bottom: auto`.
- **Обертка:** Измените способ обертки элементов Флекс в гибкий контейнер. Выберите полное отсутствие переноса (по умолчанию в браузере) с помощью `.flex-nowrap`, обертывание с помощью `.flex-wrap` или обратное обертывание с помощью `.flex-wrap-reverse`.
- **Порядок:** Измените визуальный порядок определенных элементов Флекс с помощью нескольких утилит `order`. Мы предоставляем только варианты создания элемента первым или последним, а также сброс для использования порядка DOM. Поскольку `order` принимает любое целочисленное значение от 0 до 5, добавьте собственный CSS для любых дополнительных значений.
- **Выравнивание содержимого:** Используйте утилиты `align-content` на контейнерах flexbox для выравнивания элементов Флекс вместе по поперечной оси. Выберите `start` (по умолчанию для браузера), `end`, `center`, `between`, `around` или `stretch`. Чтобы продемонстрировать эти утилиты, мы ввели в действие `flex-wrap: wrap` и увеличили количество элементов Флекс.
Внимание! Это свойство не влияет на отдельные строки элементов Флекс.

Флекс элемент 1 Флекс элемент 2 Флекс элемент 3

Флекс элемент 3 Флекс элемент 2 Флекс элемент 1

Третий флекс элемент Второй флекс элемент Первый флекс элемент

Утилиты Float (Плавающие элементы)

Float — утилиты для быстрого задания CSS-свойства `float`.

Они позволяют расположить элемент слева (`float: left`), справа (`float: right`) или отключить плавание (`float: none`).

В Bootstrap 5.3 float используется реже, чем Flexbox/Grid, но остаётся удобным инструментом для простых случаев (например, обтекание текста изображением).

Основные классы

`.float-start` — плавает влево (`float: left`)

`.float-end` — плавает вправо (`float: right`)

`.float-none` — отключает плавание (`float: none`)

Адаптивные варианты

Работают со всеми брейкпоинтами: `sm`, `md`, `lg`, `xl`, `xxl`

`.float-sm-start`

`.float-md-end`

`.float-lg-none`

Пример

`.float-start`

Слева

Текст обтекает плавающие блоки с двух сторон. На маленьких экранах поведение можно изменить с помощью адаптивных классов.

`.float-end`

Справа

```
<div class="clearfix">
  <div class="float-start">Слева</div>
  <div class="float-end">Справа</div>
</div>
```

Важно: Float убирает элемент из нормального потока, из-за чего родитель часто теряет высоту. Для очистки потока в Bootstrap 5.3 рекомендуется использовать вспомогательный класс `.clearfix` на родительском элементе.

Interaction

Утилиты для управления выделением текста и взаимодействием с указателем мыши.

Выделение текста (User Select)

.user-select-all — полностью выделяется при клике

.user-select-auto — обычное поведение браузера

.user-select-none — выделение запрещено

```
<p class="user-select-all">Полностью выделяется</p>
<p class="user-select-auto">Поведение по умолчанию</p>
<p class="user-select-none">Не выделяется</p>
```

Указатель событий (Pointer Events)

[.pe-none](#) — ссылка неактивна

[.pe-auto](#) — обычная активная ссылка

[Родитель .pe-none](#), однако у [этой ссылки](#) есть класс **pe-auto**, и по ней можно перейти.

```
<a href="#" class="pe-none">Неактивная ссылка</a>
<a href="#" class="pe-auto">Активная ссылка</a>
<p class="pe-none">Родитель с pe-none, но <a href="#" class="pe-auto">эта ссылка</a> активна</p>
```

Важно: Класс **.pe-none** отключает только события мыши. Для полной недоступности добавляйте **tabindex="-1"** и **aria-disabled="true"**.

Link

Непрозрачность ссылки

Управление прозрачностью цвета ссылки с помощью утилит `link-opacity-*`.

[link-opacity-10](#)
[link-opacity-25](#)
[link-opacity-50](#)
[link-opacity-75](#)
[link-opacity-100](#)

[Hover: 10](#)
[Hover: 50](#)

```
<a class="link-opacity-25" href="#">Opacity 25</a>  
<a class="link-opacity-10-hover" href="#">Hover opacity 10</a>
```

Подчёркивание ссылки

Цвет подчёркивания:

[primary](#) [secondary](#) [success](#) [danger](#)
[warning](#) [info](#) [light](#) [dark](#)

Смещение подчёркивания (`link-offset-*`):

[Offset 1](#)
[Offset 2](#)
[Offset 3](#)

```
<a class="link-underline-primary">Primary underline</a>  
<a class="link-offset-2">Offset 2</a>
```

Непрозрачность подчёркивания

Требуется добавить класс `.link-underline` для работы с прозрачностью.

opacity-0 25 50 75 100

Варианты при наведении + цветные ссылки

Primary link

Success link

Danger link

```
<a class="link-offset-2 link-underline link-underline-opacity-50">Underline 50</a>  
<a class="link-primary link-offset-2 link-underline-opacity-25 link-underline-opacity-100-hover">Primary</a>
```

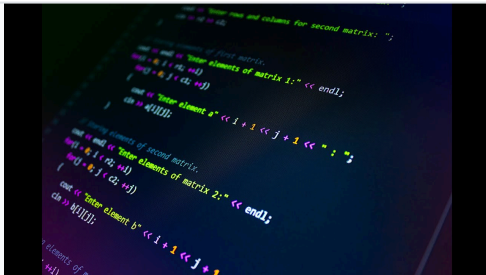
Совет по доступности: Цвет не должен быть единственным способом передачи смысла. Используйте текст или дополнительные индикаторы.

Object fit

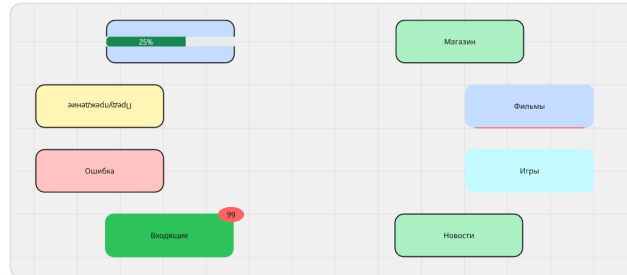
Утилиты для управления тем, как заменяемый контент (img, video) вписывается в свой контейнер.

Основные классы: `.object-fit-contain`, `.object-fit-cover`, `.object-fit-fill`, `.object-fit-scale`, `.object-fit-none`

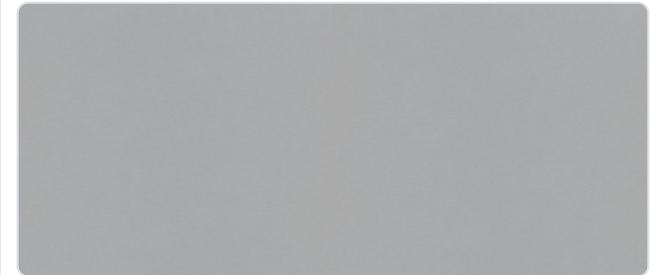
contain



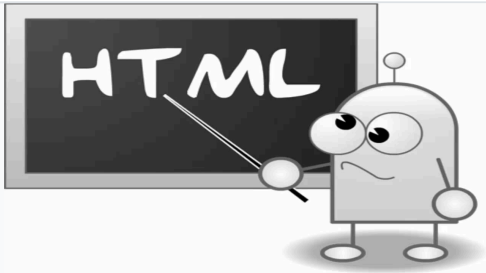
cover



fill



scale-down



none



```

```

contain — полностью помещает изображение, сохраняя пропорции (может быть пустое пространство).

cover — полностью заполняет контейнер, сохраняя пропорции (часть может обрезаться).

fill — растягивает изображение, игнорируя пропорции.

scale-down — уменьшает до нужного размера или показывает оригинал (без увеличения).

none — оригинальный размер без изменений.

Адаптивные варианты

Поддерживаются брейкпоинты: `object-fit-sm-*`, `object-fit-md-*`, `object-fit-lg-*` и т.д.

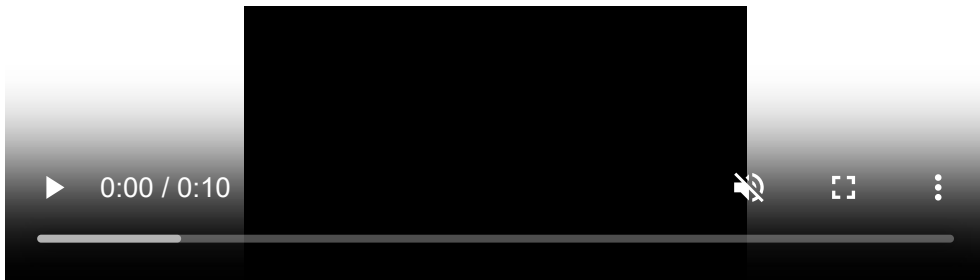


sm: contain → md: cover → lg: fill

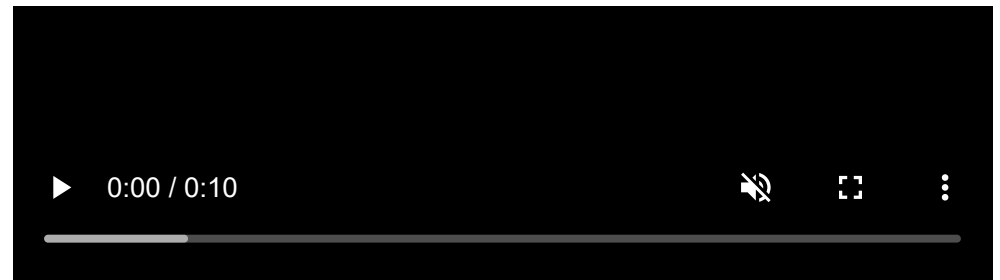
```
<img class="object-fit-sm-contain object-fit-md-cover object-fit-lg-fill w-100 h-100">
```

Object fit с видео

object-fit-contain



object-fit-cover



```
<video class="object-fit-contain w-100" style="height: 200px;"></video>  
<video class="object-fit-cover w-100" style="height: 200px;"></video>
```

Непрозрачность (Opacity)

Управляйте непрозрачностью элементов. Свойство `opacity` устанавливает уровень прозрачности, где 1 — Полностью непрозрачный, .5 — видимый на 50%, а 0 — полностью прозрачный.

Установите непрозрачность элемента с помощью утилит `.opacity-{value}`.

opacity-100

opacity-75

opacity-50

opacity-25

opacity-10

```
<div class="opacity-100">...</div>
<div class="opacity-75">...</div>
...
<div class="opacity-0">...</div>
```

Переполнение (Overflow)

Используйте эти утилиты для быстрой настройки отображения в случае, когда контент выходит за пределы элемента.

overflow

`.overflow-auto` — это пример использования для элемента с заданными размерами. По задумке,

`.overflow-auto`

`.overflow-hidden` — это пример использования для элемента с заданными размерами. Содержимое

`.overflow-hidden`

`.overflow-visible` — это пример использования для элемента с заданными размерами. Содержимое

видно за пределами контейнера.

`overflow-visible`

`.overflow-scroll` — это пример использования для элемента с заданными

`.overflow-scroll`

overflow-x

`.overflow-x-auto` — очень длинный текст, который не помещается в контейнер.



`.overflow-x-auto`

`.overflow-x-hidden` — очень длинный текст, который не помещается в контейнер.

`.overflow-x-hidden`

`.overflow-x-visible` — очень длинный текст, который не помещается в контейнер.

`.overflow-x-visible`

`.overflow-x-scroll` — всегда показывает горизонтальную прокрутку.



`.overflow-x-scroll`

ейнера

overflow-y

`.overflow-y-auto` — этот контент будет прокручиваться по вертикали. Добавим еще текст.



`.overflow-y-auto`

`.overflow-y-hidden` — контент обрезается по вертикали. Дополнительный текст не виден за пределами контейнера.

`.overflow-y-hidden`

`.overflow-y-visible` — контент виден за пределами контейнера. Этот текст выходит за границы блока.

Да да, действительно выходит. Не верите? а вы поверьте!

`.overflow-y-visible`

`.overflow-y-scroll` — всегда показывает вертикальную прокрутку.



`.overflow-y-scroll`

Сводка

.overflow — управляет поведением контента, который выходит за границы элемента.

.overflow-x — управляет только горизонтальным переполнением.

.overflow-y — управляет только вертикальным переполнением.

```
<div class="overflow-auto">...</div>
<div class="overflow-hidden">...</div>
<div class="overflow-visible">...</div>
<div class="overflow-scroll">...</div>
```

Чаще всего используются `.overflow-auto` и `.overflow-hidden`.

Позиция (Position)

Утилиты для быстрой настройки свойства `position` элемента.

Основные классы: `.position-static`, `.position-relative`, `.position-absolute`, `.position-fixed`, `.position-sticky`,

`.position-static`

`.position-relative`

`.position-absolute`

`.position-sticky`

Прокрути этот контейнер, чтобы увидеть sticky элемент в действии

Lorem ipsum dolor sit amet...

```
<div class="position-sticky">...</div>
```

Описание классов

static — стандартное положение в потоке (по умолчанию)

relative — позиционируется относительно своего обычного места

absolute — позиционируется относительно ближайшего позиционированного родителя

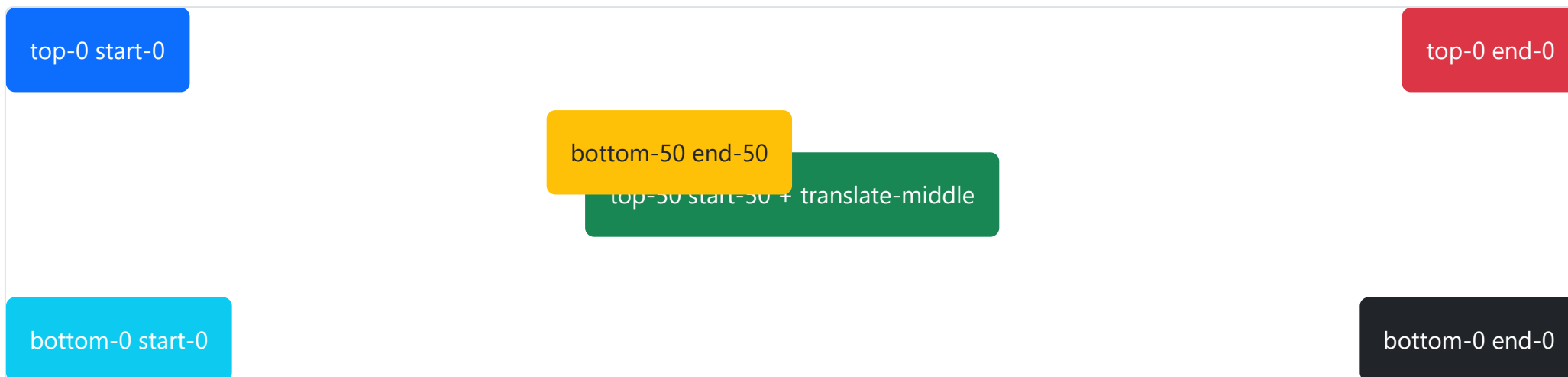
fixed — фиксируется относительно окна браузера (viewport)

sticky — прилипает при прокрутке (гибрид relative + fixed)

Расстановка элементов

Утилиты быстрого позиционирования позволяют легко размещать элементы внутри контейнера с помощью комбинаций `top-*`, `start-*`, `end-*` и `bottom-*`. Все они работают только вместе с классом `position-absolute` (или `position-fixed`).

Полезные комбинации: `top-0 start-0`, `top-0 end-0`, `bottom-0 start-0`, `bottom-0 end-0`, `top-50 start-50 translate-middle` (центрирование).



```
<div class="position-relative" style="height: 320px;">
  <div class="position-absolute top-0 start-0">top-0 start-0</div>
  <div class="position-absolute top-0 end-0">top-0 end-0</div>
  <div class="position-absolute top-50 start-50 translate-middle">Центр</div>
  <div class="position-absolute bottom-0 start-0">bottom-0 start-0</div>
  <div class="position-absolute bottom-0 end-0">bottom-0 end-0</div>
</div>
```

Полезный совет: Для точного центрирования элемента используйте комбинацию `top-50 start-50 translate-middle`. Класс `translate-middle` сдвигает элемент на половину его собственной ширины и высоты.

Центрирование элементов

Для удобного центрирования абсолютно позиционированных элементов используйте вспомогательные классы `translate-middle`, `translate-middle-x` и `translate-middle-y`.

translate-middle сдвигает элемент на 50% его ширины и высоты, что позволяет легко центрировать его по горизонтали и вертикали.

top-0 start-0

top-0 start-50 + translate-middle-x

top-0 end-0

top-50 start-0 + translate-middle-y

top-50 start-50 + translate-middle

top-50 end-0 + translate-middle-y

bottom-0 start-0

bottom-0 start-50 + translate-middle-x

bottom-0 end-0

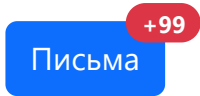
```
<div class="position-relative" style="height: 280px;">
  <div class="position-absolute top-50 start-50 translate-middle">Центр по обеим осям</div>
  <div class="position-absolute top-0 start-50 translate-middle-x">Центр по горизонтали</div>
  <div class="position-absolute top-50 start-0 translate-middle-y">Центр по вертикали</div>
</div>
```

Класс `translate-middle` — самый удобный способ центрирования абсолютно позиционированных элементов. Он работает за счёт `transform: translate(-50%, -50%);`.

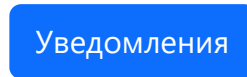
Примеры

Классы позиционирования для создания маркеров, бейджей и шаговых индикаторов.

1. МАРКЕР НА КНОПКЕ



2. ПУСТОЕ УВЕДОМЛЕНИЕ



3. ШАГОВЫЙ ИНДИКАТОР



```
<!-- 1. Маркер с числом -->
```

```
<button class="position-relative">...
```

```
  <span class="position-absolute top-0 start-100 translate-middle badge rounded-pill bg-danger">+99</span>
```

```
</button>
```

```
<!-- 2. Маркер-точка -->
```

```
<span class="position-absolute top-0 start-100 translate-middle badge border border-light rounded-circle bg-danger p-2"></span>
```

```
<!-- 3. Элемент шага (центрирование на линии) -->
```

```
<button class="position-absolute top-0 start-50 translate-middle btn btn-sm btn-primary rounded-pill">2</button>
```

Тени (Shadows)

Утилиты теней позволяют быстро добавлять или удалять эффект `box-shadow` к элементам. Тени на компонентах по умолчанию отключены, но их можно включить через переменную Sass `$enable-shadows`.

- **Доступные классы:** `.shadow-none`, `.shadow-sm`, `.shadow`, `.shadow-lg`.
- **Особенности:** Значения теней хранятся в CSS-переменных, что обеспечивает гибкость. Классы используют `!important`.

`.shadow-none` — без тени

`.shadow-sm` — малая тень

`.shadow` — стандартная тень

`.shadow-lg` — большая тень

```
<div class="shadow-sm p-3 bg-body-tertiary rounded">Малая тень</div>
```

Размеры (Sizing)

Утилиты размеров позволяют быстро задавать ширину и высоту элементов относительно родителя или viewport. Поддерживаются процентные значения, **auto** и единицы viewport.

- **Ширина:** **.w-25**, **.w-50**, **.w-75**, **.w-100**, **.w-auto**.
- **Высота:** аналогично — **.h-25** ... **.h-100**, **.h-auto**.
- **Максимальные размеры:** **.mw-100** (max-width: 100%), **.mh-100**.
- **Viewport:** **.vw-100**, **.vh-100**, **.min-vw-100**, **.min-vh-100**.
- **Адаптивность:** классы работают со всеми брейкпоинтами.

.w-25

.w-50

.w-75

.w-100

```
<div class="w-50">Ширина 50%</div>
<div class="h-100">Высота 100%</div>
<div class="vw-100">Ширина 100vw</div>
```

Отступы (Spacing)

Утилиты отступов — один из самых используемых наборов в Bootstrap. Они позволяют быстро управлять `margin` и `padding` для всех сторон элемента, отдельных осей или конкретных сторон.

- **Сокращения:** `m-` — margin, `p-` — padding.
- **Стороны:** `t` (top), `b` (bottom), `s` (start — лево в LTR), `e` (end — право в LTR), `x` (горизонтальные стороны), `y` (вертикальные стороны), без буквы — все четыре стороны.
- **Размеры:** от `0` (0) до `5` (3rem), а также `auto` (только для margin).
- **Отрицательные margins:** включаются через переменную Sass `$enable-negative-margins: true`. Используется префикс `n` — например, `.mt-n3`.
- **Gap:** утилиты `.gap-*`, `.row-gap-*` и `.column-gap-*` для управления расстоянием между элементами в flex и grid-контейнерах.
- **Центрирование:** `.mx-auto` — горизонтальное центрирование блочного элемента фиксированной ширины.

Адаптивность отступов

Все утилиты отступов поддерживают адаптивные варианты. Класс без префикса брейкпоинта применяется ко всем размерам экрана (от `xs`). Чтобы задать отступ только начиная с определённого брейкпоинта, добавьте префикс: `sm`, `md`, `lg`, `xl` или `xxl`.

Примеры адаптивного поведения:

- `.ms-2 ms-md-4` — на мобильных отступ слева 0.5rem, на `md` и шире — 1.5rem.
- `.p-3 p-lg-5` — базовый padding 1rem, на больших экранах — 3rem.
- `.mt-0 mt-xl-5` — убрать отступ сверху на маленьких экранах и добавить большой отступ на `xl`.

Этот блок имеет адаптивные внутренние отступы:

`p-3` на мобильных,

`p-md-4` на средних экранах,

`p-xl-5` на очень больших.

Примеры использования

.mt-3 — отступ сверху

.mx-auto —
горизонтальное
центрирование

.ms-4 — отступ слева (start)

Элемент 1

Элемент 2

Элемент 3

Элемент 4

.d-grid .gap-3 — одинаковый зазор между элементами

```
<!-- Базовые примеры -->
<div class="mt-3">Отступ сверху</div>
<div class="px-4">Горизонтальные отступы</div>
<div class="p-5">Отступы со всех сторон</div>

<!-- Адаптивные -->
<div class="ms-2 ms-md-5">Отступ слева меняется по брейкпоинтам</div>
<div class="mx-auto" style="width: 300px;">Центрированный блок</div>

<!-- Отрицательный margin -->
<div class="mt-n3">Отрицательный отступ сверху</div>

<!-- Gap -->
<div class="d-flex gap-4">...</div>
<div class="d-grid gap-3 row-gap-5">...</div>
```

Текст (Text)

Утилиты текста предоставляют полный контроль над оформлением текстового содержимого: выравнивание, перенос слов, регистр, размер, вес шрифта, межстрочный интервал и многое другое.

- **Выравнивание:** `.text-start`, `.text-center`, `.text-end` + адаптивные варианты (`.text-md-center` и т.д.).
- **Перенос текста:** `.text-wrap`, `.text-nowrap`, `.text-break`.
- **Регистр:** `.text-lowercase`, `.text-uppercase`, `.text-capitalize`.
- **Размер шрифта:** `.fs-1` ... `.fs-6` (соответствует `h1–h6`).
- **Вес шрифта:** `.fw-bold`, `.fw-semibold`, `.fw-medium`, `.fw-normal`, `.fw-light`, `.fw-lighter`.
- **Стиль:** `.fst-italic`, `.fst-normal`.
- **Межстрочный интервал:** `.lh-1`, `.lh-sm`, `.lh-base`, `.lh-lg`.
- **Моноширинный шрифт:** `.font-monospace`.
- **Декорация:** `.text-decoration-none`, `.text-decoration-underline`, `.text-decoration-line-through`.
- **Сброс стилей:** `.text-reset`.

Примеры использования

`.text-start` — выравнивание по левому краю

`.text-center` — выравнивание по центру

`.text-end` — выравнивание по правому краю

На мобильных — по центру (`.text-center`),
на `md` — по левому краю (`.text-md-start`),
на `lg` и больше — по правому краю (`.text-lg-end`).

`.text-nowrap` — текст не переносится

`.text-wrap` — обычный
перенос (по
умолчанию)

ОченьДлинноеСловоКо
тороеНеПомещаетсяВК
онтейнерИДолжноРазо
рваться.`.text-break`

`.fs-1 .fw-bold`

`.fs-2 .fw-semibold`

`.fs-3 .fw-medium`

`.fs-4 .fw-normal`

`.fs-5 .fw-light`

`.fs-6 .fw-lighter .fst-italic`

.lh-1 Это длинный абзац, демонстрирующий, как наши утилиты влияют на межстрочный интервал элемента. Классы применяются к самому элементу или иногда к родительскому элементу. Эти классы можно настраивать по мере необходимости с помощью нашего API утилит.

.lh-sm Это длинный абзац, демонстрирующий, как наши утилиты влияют на межстрочный интервал элемента. Классы применяются к самому элементу или иногда к родительскому элементу. Эти классы можно настраивать по мере необходимости с помощью нашего API утилит.

.lh-base Это длинный абзац, демонстрирующий, как наши утилиты влияют на межстрочный интервал элемента. Классы применяются к самому элементу или иногда к родительскому элементу. Эти классы можно настраивать по мере необходимости с помощью нашего API утилит.

.lh-lg Это длинный абзац, демонстрирующий, как наши утилиты влияют на межстрочный интервал элемента. Классы применяются к самому элементу или иногда к родительскому элементу. Эти классы можно настраивать по мере необходимости с помощью нашего API утилит.

ТЕКСТ В ВЕРХНЕМ РЕГИСТРЕ — .TEXT-UPPERCASE

текст в нижнем регистре — .text-lowercase

Каждое Слово С Большой Буквы — .Text-Capitalize

Подчёркнутый текст — .text-decoration-underline

~~Зачёркнутый текст~~ — .text-decoration-line-through

Ссылка без подчёркивания — .text-decoration-none

Это моноширинный текст — .font-monospace

Текст со сброшенными цветом и оформлением — .text-reset

<!-- Основные примеры -->

<p class="text-center fw-bold fs-3">Заголовок по центру</p>

<p class="text-md-end">Адаптивное выравнивание</p>

<p class="text-nowrap">Без переноса</p>

<p class="fs-1 fw-bold lh-lg">Большой жирный текст с большим интервалом</p>

<p class="text-uppercase text-decoration-underline">Верхний регистр и подчёркивание</p>

Вертикальное выравнивание (Vertical align)

Утилиты вертикального выравнивания работают с inline-, inline-block- и табличными элементами, изменяя свойство `vertical-align`.

- **Классы:** `.align-baseline`, `.align-top`, `.align-middle`, `.align-bottom`, `.align-text-top`, `.align-text-bottom`.
- **Особенности:** Не работают с блочными элементами (для них используйте flex).

baseline top middle bottom text-top text-bottom

```
<span class="align-middle">Выравнивание по середине</span>
```

Видимость (Visibility)

Утилиты видимости управляют свойством `visibility`, не влияя на поток документа и занимаемое место (в отличие от `display: none`).

- **Классы:** `.visible` и `.invisible`.
- **Особенности:** `.invisible` скрывает элемент визуально и для экранных читалок, но оставляет пространство.

Видимый элемент

<-- тут невидимый элемент, но он занимает место

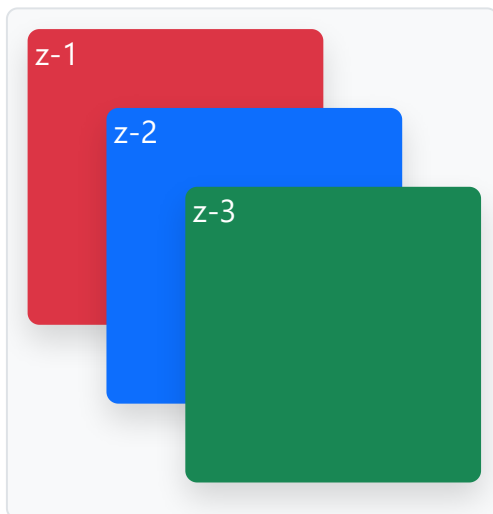
```
<div class="invisible">Скрытый, но занимающий место</div>
```

Z-index

Утилиты **z-index** позволяют быстро управлять порядком наложения (перекрытия) элементов друг относительно друга. Чем больше значение z-index, тем «выше» находится элемент на странице.

Важно: утилиты z-index работают **только** с элементами, у которых свойство `position` отличается от `static` (например, `relative`, `absolute`, `fixed` или `sticky`).

- **Доступные классы:** `.z-n1` (z-index: -1), `.z-0` (0), `.z-1` (1), `.z-2` (2), `.z-3` (3).
- Это низкоуровневые утилиты. Более высокие значения используются компонентами Bootstrap (модальные окна, тултипы и др.).



```
<div class="position-relative" style="height: 260px;">
  <!-- Самый нижний -->
  <div class="position-absolute top-0 start-0 z-n1 bg-warning p-3">z-n1</div>

  <!-- Самый верхний -->
  <div class="position-absolute top-50 start-50 translate-middle z-3 bg-primary p-3">z-3</div>
</div>
```

Зелёный блок (`.z-3`) перекрывает все остальные.

Если требуется больше уровней наложения, вы можете расширить утилиты через API Bootstrap или задать нужное значение напрямую в своём CSS.